



RECOMMENDER SYSTEM

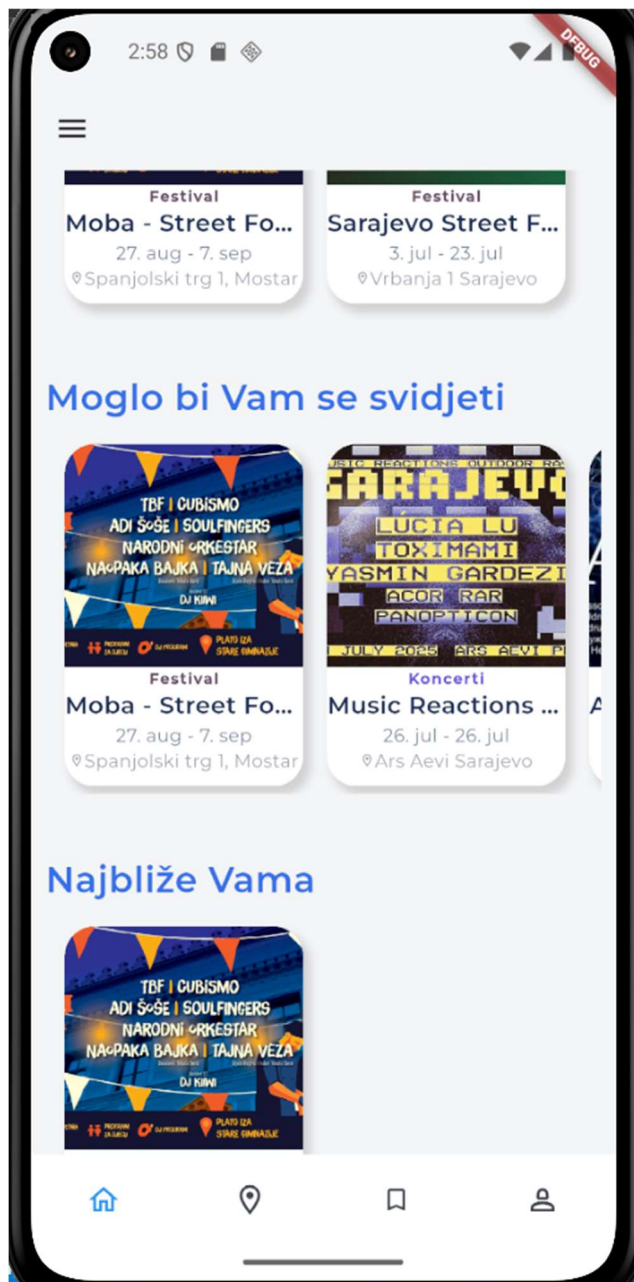
eventsApp

IB190024
Hana Fejzić

Recommender system radi preporuku događaja user-ima za koje misli da bi im mogli biti interesantni. Recommender system preporuka se može pronaći na home page ekranu u sekciji 'Moglo bi Vam se svidjeti'.

Putanja za recommender system na UI:

eventsApp/UI/eventsappusers/lib/screens/home_screen



Opis implementacije

U aplikaciji eventsApp je implementiran user-based collaborative filtering, što znači da se useru preporučuju događaji na koje su useri sa sličnim preferencijama pozitivno reagovali. U koliko su korisnici slični, znači da će imati iste potrebe.

Ono što se uzima u obzir prilikom računanja sličnosti korisnika je implicitni feedback, odnosno događaji na koje su useri pozitivno reagovali - gledali, spremili, komentarisali, pratili, kupovali karte...

Podaci za treniranje algoritma se čitaju iz baze. Svaka interakcija ima određenu težinu, nije isto kupovina karte i pregled događaja. Zbog toga sam uvela još jednu varijablu koja će se zvati interactionScore koja vrednuje različite akcije po težini.

Trening će se pokrenuti sa pokretanjem aplikacije. Naravno, želimo da nam je model up-to-date i da se češće trenira te je iz tog razloga dok god je aplikacija pokrenuta, zakazan scheduler koji okida treniranje modela svaki dan u ponoć.

Putanja za recommender system:

eventsApp/eventsApp.Services/RecommenderSystemServiceImpl.cs

Prvo se prikuplja sav implicitni feedback koji je sačuvan u raličitim tabelama u bazi: HistorijaPregleda, Narudzbe, Savings, Komentari. U ovom koraku se radi bodovanje različitih akcija po principu:

Akcija	Interaction score
Bought Tickets	5.0
Saved Event	4.0
Commented	3.0
Viewed Event	1.0

```
1 reference | hana_fejzic, 16 hours ago | 2 authors, 2 changes
private async Task<List<EventInteraction>> GetData()
{
    var data = new List<EventInteraction>();

    var savedEvents = await _context.Savings
        .Select(se => new EventInteraction
        {
            UserId = (uint)se.KorisnikId,
            EventId = (uint)se.DogadjajId,
            InteractionScore = 4.0f
        }).ToListAsync();
    data.AddRange(savedEvents);
}
```

```

var viewedEvents = await _context.HistorijaPregleda
    .Select(ve => new EventInteraction
    {
        UserId = (uint)ve.KorisnikId,
        EventId = (uint)ve.DogadjajId,
        InteractionScore = 1.0f
    }).ToListAsync();
data.AddRange(viewedEvents);

var ticketPurchases = await _context.Narudzbis.Include(n => n.NarudzbaStavkes).ThenInclude(ns => ns.TipKarte)
    .Where(n => n.NarudzbaStavkes.Any()).Select(n => new EventInteraction
    {
        UserId = (uint)n.KorisnikId,
        EventId = (uint)(n.NarudzbaStavkes.First().TipKarte.DogadjajId ?? 0),
        InteractionScore = 5.0f
    }).ToListAsync();
data.AddRange(ticketPurchases);

var commentedEvents = await _context.Komentaris
    .Select(c => new EventInteraction
    {
        UserId = (uint)c.KorisnikId,
        EventId = (uint)c.DogadjajId,
        InteractionScore = 3.0f
    }).ToListAsync();
data.AddRange(commentedEvents);

return data;
}

```

Koristimo MLContext da kreiramo model te podesimo potrebne parametre za trening. Dijelimo skup podataka na 80% skup za trening i 20% skup za test. Nakon toga radimo evaluaciju modela i spašavamo ga u bazu podataka. Istrenirani model se čuva u bazi u tabeli TrainedModels kako bi mu kasnije mogli lakše pristupiti da ne moramo svaki put trenirati model.

```

4 references | Hana Fejzic, 19 days ago | 1 author, 2 changes
public async Task<ITransformer> CreateModel()
{
    _logger.LogInformation("Model creation started...");

    var mlContext = new MLContext();

    var interactions = await GetData();

    if (!interactions.Any()) {
        _logger.LogWarning("No interaction data available. Skipping model training.");
        return null;
    }

    var trainData = mlContext.Data.LoadFromEnumerable(interactions);
    var trainTestSplit = mlContext.Data.TrainTestSplit(trainData, testFraction: 0.2);
    var testData = trainTestSplit.TestSet;

    var options = new MatrixFactorizationTrainer.Options
    {
        MatrixColumnIndexColumnName = "UserIdEncoded",
        MatrixRowIndexColumnName = "EventIdEncoded",
        LabelColumnName = "InteractionScore",
        LossFunction = MatrixFactorizationTrainer.LossFunctionType.SquareLossOneClass,
        Alpha = 0.01,
        Lambda = 0.025,
        NumberOfIterations = 20,
        ApproximationRank = 20
    };
}

```

```

var pipeline = mlContext.Transforms.Conversion.MapValueToKey(outputColumnName: "UserIdEncoded", inputColumnName: "UserId")
    .Append(mlContext.Transforms.Conversion.MapValueToKey(outputColumnName: "EventIdEncoded", inputColumnName: "EventId"))
    .Append(mlContext.Recommendation().Trainers.MatrixFactorization(options));

var model = pipeline.Fit(trainTestSplit.TrainSet);
if (model == null)
{
    throw new Exception("Model training failed, resulting model is null.");
}

_logger.LogInformation("Model creation ended");

_logger.LogInformation("Model evaluation started...");
var prediction = model.Transform(testData);
var metrics = mlContext.Regression.Evaluate(prediction, labelColumnName: "InteractionScore", scoreColumnName: "Score");

Console.WriteLine("Root Mean Squared Error : " + metrics.RootMeanSquaredError.ToString());
Console.WriteLine("RSquared: " + metrics.RSquared.ToString());
_logger.LogInformation("Model evaluation ended.");

_logger.LogInformation("Save model started...");
await SaveModelToDatabase(mlContext, trainData.Schema, model);
_logger.LogInformation("Save model ended.");
return model;

```

Prije samog predlaganja događaja korisniku, implementirana je validacija da li korisnik postoji u sistemu, kao i da li ima dovoljno podataka da bi se događaj mogao predložiti. Dovoljno je da je user barem jednom ušao u neki događaj da bi mu se drugi događaji mogli predlagati, a ako nema zapisa u njegovoj historiji pregleda, onda mu se predlažu top 3 najpopularnija događaja.

Za sistem preporuke dohvata se iz baze posljednji istrenirani model i koristi se za kreiranje prediction engine-a. U obzir spadaju svi događaji koji su u statusu ACTIVE, što znači da su odobreni od strane administratora da se objave i koji vremenski traju u ovom trenutku ili će se desiti u budućnosti. Također, gleda se i da korisnik nije već prije kupio kartu za taj događaj.

Za svaki od mogućih događaja predviđa se prediction score. Događaji sa najvišim prediction score imaju najveću šansu da će se svidjeti korisniku te se uzimaju top 3 događaja za preporuku korisniku.

1 reference | hana_fejzic, 1 day ago | 2 authors, 2 changes

```
private async Task<List<Database.Dogadjaji>> RecommendEvents(MLContext mlContext, ITransformer model, int userId)
{
    var predictionEngine = mlContext.Model.CreatePredictionEngine<EventInteraction, EventPrediction>(model);

    var possibleEvents = await _context.Dogadjajis.Include(d=>d.Kategorija).Include(d => d.TipKartes).
        ThenInclude(t => t.NarudzbaStavkes).ThenInclude(n => n.Narudzba)
        .Where(d => d.Status == "ACTIVE" &&
            !d.TipKartes.Any(t => t.NarudzbaStavkes.Any(n => n.Narudzba.KorisnikId == userId))).ToListAsync();
    if (!possibleEvents.Any()) throw new Exception("Ne postoje događaji za predložiti");

    var predictionList = new List<Tuple<Database.Dogadjaji, float>>();

    foreach (var e in possibleEvents)
    {
        var input = new EventInteraction { UserId = (uint)userId, EventId = (uint)e.DogadjajId };
        var prediction = predictionEngine.Predict(input);
        Console.WriteLine($"User id {userId} event prediction : event id {e.DogadjajId}\nScore: {prediction.Score}");
        predictionList.Add(new Tuple<Database.Dogadjaji, float>(e, prediction.Score));
    }

    return predictionList
        .OrderByDescending(p => p.Item2)
        .Take(3)
        .Select(p => p.Item1)
        .ToList();
}
```